Separation-Driven Graph Coloring: A Unified Scoring Framework for Constraint-Aware Optimization

Mann Lohchab
Department of Mathematics
SRM University Delhi NCR Haryana 131039
mannlohchab@gmail.com

Dr. R.P Yadav
Department of Mathematics
SRM University Delhi NCR Haryana 131039
rpmaths.ism@gmail.com

***Abstract*—Graph coloring is a fundamental NP-hard combinatorial optimization problem with applications in scheduling, wireless communication, frequency assignment, and resource allocation. Classical heuristics such as greedy coloring and DSATUR primarily rely on local structural information, which can lead to inefficient color utilization in structurally complex graphs. This paper introduces Separation-Driven Graph Coloring (SDGC), a unified scoring-based heuristic framework that formulates color assignment as an optimization problem. Each candidate assignment is evaluated using a composite scoring function integrating three components: a separation-aware metric promoting global color dispersion, a conflict penalty enforcing constraint satisfaction, and a regularization term discouraging unnecessary color introduction. A single scoring rule governs both the initial assignment and the repair phase, ensuring consistency throughout the optimization process. Experimental evaluation on standard DIMACS benchmark graphs demonstrates that SDGC achieves competitive performance on sparse and medium-density instances while exhibiting reduced effectiveness on dense graphs, where separation-based reasoning loses discriminative power under high conflict pressure. The results suggest that integrating global structural awareness with color regularization provides a technically sound and practically effective direction for heuristic graph coloring.**

***Index Terms*—Graph Coloring, Combinatorial Optimization, Heuristic Optimization, Constraint-Aware Optimization, Graph Algorithms, NP-Hard Problems, Separation-Driven Optimization**

I. Introduction

Graph coloring is a classical combinatorial optimization problem in which vertices of a graph are assigned colors such that no two adjacent vertices share the same color. The minimum number of colors required is the chromatic number $\chi(G)$. Determining $\chi(G)$ is NP-hard in general [3], and practical algorithms therefore rely on heuristic or approximation strategies.

Greedy coloring and DSATUR [2] are among the most widely adopted heuristics due to their low computational overhead. Both methods rely exclusively on local information: adjacency structure and, in the case of DSATUR, the number of distinct colors in a vertex's neighborhood. While these approaches are effective across a broad class of instances, they do not account for the global distribution of colors in the graph, which can result in premature color commitments that reduce flexibility for subsequent vertices.

A further limitation of existing approaches is that color assignment is treated as a constraint satisfaction problem: the objective is feasibility, and no explicit mechanism governs color economy during the assignment process. This can lead to unnecessary color proliferation, particularly in dense graph regions where conflict pressure is high.

We introduce Separation-Driven Graph Coloring (SDGC), a framework that addresses these limitations by formulating color assignment as a scoring maximization problem. The scoring function simultaneously accounts for global color dispersion via a BFS-based separation metric, constraint feasibility via a conflict penalty, and color economy via a regularization term that penalizes the introduction of new colors. A single unified rule governs all assignment decisions, including those made during the repair phase.

A. Contributions

The primary contributions of this work are as follows:

- A BFS-based separation metric that quantifies global color distribution beyond the immediate neighborhood of each vertex.
- A color regularization term that penalizes unnecessary color introduction, reducing total color usage relative to unregularized separation-driven assignment.
- A unified scoring rule applied consistently across the initial assignment and repair phases, eliminating heuristic inconsistency between stages.
- An experimental evaluation on standard DIMACS benchmark graphs with direct comparison against greedy coloring and DSATUR, including analysis of performance across sparse, medium-density, and dense instances.

II. Related Work

Greedy coloring with degree-based vertex ordering [4] and the DSATUR algorithm [2] remain foundational references for practical graph coloring. DSATUR selects at each step the uncolored vertex with the highest number of distinct colors

among its neighbors (saturation degree), breaking ties by degree. This ordering strategy consistently produces competitive colorings and serves as a strong benchmark for heuristic evaluation [5].

Local search methods, including tabu search [15] and simulated annealing, improve an initial feasible coloring by iteratively recoloring vertices to reduce color count. These methods can achieve near-optimal results but require a feasible starting solution and may incur significant computational cost.

Population-based approaches such as hybrid evolutionary algorithms combine crossover operators with local search to explore the solution space more broadly. While such methods can attain strong results on benchmark instances, they introduce substantial complexity and are less amenable to straightforward generalization.

Recent work in graph coloring has increasingly focused on hybrid heuristics, learning-guided search, and adaptive optimization strategies for large benchmark instances. Contemporary approaches combine local search, reinforcement-guided selection, and evolutionary optimization to improve coloring quality under complex constraint environments. These developments highlight continued interest in balancing solution quality, runtime efficiency, and structural adaptability in heuristic coloring frameworks.

The present work differs from these approaches in that it does not rely on a separate feasibility phase followed by optimization. Instead, SDGC integrates constraint satisfaction and color economy into a single scoring function applied uniformly at every assignment step, providing a lightweight mechanism for globally informed heuristic coloring.

III. Methodology

A. Scoring Framework

Let $G = (V, E)$ be the input graph and $\mathcal{C}$ a set of available colors. For each vertex $v \in V$ and each candidate color c, we define:

$$\text{score}(v, c) \;=\; \alpha \cdot \text{sep}(v, c) \;-\; \beta \cdot \text{conf}(v, c) \;-\; \gamma \cdot \text{new}(c) \quad (1)$$

where:

- $\text{sep}(v, c) \in [0, k]$ measures the graph-distance separation between v and the nearest vertex already assigned color c, computed via bounded BFS to depth k. Higher values indicate that color c is spatially dispersed from v, which is desirable.
- $\text{conf}(v, c) \geq 0$ quantifies the total constraint violation incurred by assigning color c to v, including hard adjacency penalties and any applicable soft penalties.
- $\text{new}(c) \in \{0, 1\}$ is an indicator variable equal to 1 if color c has not yet been assigned to any vertex in the current partial coloring, and 0 otherwise. This term acts as a regularizer penalizing unnecessary color introduction.
- $\alpha, \beta, \gamma \geq 0$ are scalar weighting parameters controlling the relative influence of each component.

The color assigned to v is selected by:

$$C(v) \;=\; \arg\max_{c \,\in\, L(v)} \; \text{score}(v, c) \quad (2)$$

where $L(v)$ denotes the candidate color set for v (defined in Section III-E). This formulation transforms graph coloring from a constraint satisfaction problem into a maximization problem over a structured objective.

The theoretical complexity of a single assignment pass is $O(|V| \cdot |L| \cdot (|V| + |E|))$, where $|L|$ is the maximum candidate set size. In practice, runtime is substantially lower for two reasons. First, $L(v)$ is restricted to currently used colors plus one new color, keeping $|L|$ small relative to the total color palette. Second, BFS terminates as soon as the nearest same-colored vertex is found or the depth bound k is reached, limiting the search to a local subgraph rather than traversing the entire graph.

B. Separation Metric

For vertex v and color c, $\text{sep}(v, c)$ is computed by performing a BFS from v, up to depth k, and returning the hop distance to the nearest vertex u already assigned color c:

$$\text{sep}(v, c) = \begin{cases} h(v, u) & \text{if } \exists\, u \text{ with } C(u) = c \text{ within depth } k \\ k & \text{otherwise} \end{cases} \quad (3)$$

where $h(v, u)$ is the hop distance between v and u in G. When no same-colored vertex exists within depth k, the metric is saturated at k, reflecting maximal separation. This metric encourages the reuse of colors in regions of the graph that are structurally distant from existing assignments, promoting global dispersion without requiring full shortest-path computation.

C. Conflict Penalty

The conflict term $\text{conf}(v, c)$ aggregates penalties from all constraint types active in the problem instance:

- *Hard adjacency constraints*: a penalty of 5 is applied for each neighbor of v already assigned color c. Violations of this constraint render a coloring infeasible.
- *Soft distance-based constraints*: graded penalties are applied to vertex pairs that satisfy a proximity condition but whose assignment does not constitute a hard conflict. The penalty magnitude decreases with increasing distance, reflecting the diminishing severity of interference.
- *Domain-specific constraints*: additional penalties can be incorporated for problem-specific requirements such as minimum frequency separation or list coloring restrictions, without modifying the scoring structure.

D. Color Regularization

Without explicit color economy, separation-driven scoring can exhibit a tendency toward color proliferation in high-density graph regions. When many candidate colors from the existing palette carry high conflict penalties, the unregularized scoring function assigns a high score to a new color, since a freshly introduced color has zero conflicts and maximum

separation by default. If this mechanism operates unchecked, the algorithm introduces new colors more aggressively than the graph structure requires, inflating the total color count beyond what is achieved by conflict-avoiding reuse.

The term $\gamma \cdot \text{new}(c)$ in (1) addresses this by imposing a fixed cost on every new color introduction. This acts as a regularization term in the objective: it biases the algorithm toward reusing an existing color whenever the marginal gain in separation or conflict reduction from a new color does not exceed γ. The parameter γ controls the strength of this regularization. A larger value enforces stricter color economy but may increase constraint violations if feasible reuse options are limited. A smaller value relaxes the penalty, permitting new colors when the conflict landscape makes reuse structurally inadvisable. Appropriate calibration of γ is therefore problem-dependent and represents a trade-off between color count minimization and constraint feasibility.

E. Unified Decision Process

At each assignment step, the candidate color set is defined as:

$$L(v) \;=\; \mathcal{C}_{\text{used}} \;\cup\; \{c_{\text{new}}\} \tag{4}$$

where $\mathcal{C}_{\text{used}}$ is the set of colors currently present in the partial coloring and c_{new} is a single designated new color not yet used. Restricting the candidate set in this way keeps the per-vertex search space compact and ensures that the new-color penalty acts as an effective gate on color introduction.

The same scoring rule (1) and candidate set (4) are used during both the initial assignment pass and each iteration of the repair phase. This uniformity avoids the heuristic drift that arises when different decision criteria govern different stages, and it simplifies the extension of SDGC to new constraint types, which require only an update to the conflict penalty function.

Algorithm 1 Separation-Driven Graph Coloring (SDGC)

```
Initialize all vertices as UNCOLORED
Order vertices by decreasing degree
for each vertex v in order do
    L(v) ← C_used ∪ {c_new}
    for each color c ∈ L(v) do
        Compute sep(v, c) via bounded BFS (depth k)
        Compute conf(v, c) from active constraints
        new(c) ← 1 if c ∉ C_used, else 0
        score(v, c) ← α·sep(v, c) − β·conf(v, c) − γ·new(c)
    end for
    C(v) ← arg max_{c∈L(v)} score(v, c)
    Update C_used if C(v) is a new color
end for
// Repair Phase (3 iterations)
for r = 1 to 3 do
    for each vertex v do
        L(v) ← C_used ∪ {c_new}
        Recompute C(v) using scoring rule (1)
        Update C_used accordingly
    end for
end for
return C
```

IV. Algorithm

V. Experimental Evaluation

A. Overview

We evaluate SDGC on three standard DIMACS benchmark graphs and compare its coloring performance against two baselines: greedy coloring with degree-based vertex ordering [4] and DSATUR [2]. All experiments use fixed parameters $\alpha = 3$, $\beta = 10$, $\gamma = 8$, BFS depth $k = 3$, and 3 repair iterations. A coloring is considered valid if it contains no hard adjacency violations. Runtime is measured in seconds on a single-core execution environment.

B. Benchmark Description

The benchmark suite includes sparse, medium-density, and dense DIMACS graph instances commonly used in heuristic graph coloring evaluation.

- **myciel4**: A sparse Mycielski graph with 23 vertices and 71 edges, with chromatic number $\chi = 5$.
- **anna**: A structured benchmark graph with 138 vertices and 493 edges. The graph is moderately sparse and is frequently used as a reference instance for evaluating heuristic coloring behavior.
- **david**: A DIMACS benchmark graph containing 87 vertices and 406 edges, representing a medium-density structured instance.
- **huck**: A sparse benchmark graph with 74 vertices and 301 edges.
- **DSJC125.1**: A random graph with 125 vertices and approximately 736 edges from the DIMACS challenge suite.
- **DSJC250.1**: A larger sparse random graph with 250 vertices and approximately 3,218 edges.
- **DSJC250.5**: A denser random graph with 250 vertices and approximately 15,668 edges, representing a substantially more difficult constraint environment.
- **queen15_15**: A queen's graph on a 15×15 chessboard with 225 vertices and 5,180 edges.

C. Results

All three algorithms produce valid colorings across every benchmark. Reported color counts represent single-run results under deterministic execution with fixed random seeds; all three methods are deterministic given a fixed vertex ordering, so results are reproducible without averaging over multiple trials.

D. Discussion

The results reflect behavior that is consistent with the structural properties of the scoring function across different density regimes.

TABLE I
DIMACS BENCHMARK COLORING RESULTS

Graph	Vertices	Density	SDGC	Greedy	DSATUR	SDGC Time (s)	DSATUR Time (s)
myciel4	23	Sparse	5	6	5	0.03	0.01
anna	138	Sparse	11	12	11	0.19	0.07
huck	74	Sparse	11	12	11	0.11	0.04
david	87	Medium	12	13	11	0.15	0.05
DSJC125.1	125	Medium	18	21	17	0.41	0.12
DSJC250.1	250	Medium	30	33	28	1.24	0.39
DSJC250.5	250	Dense	46	44	41	3.81	0.94
queen15_15	225	Dense	28	26	24	1.87	0.51

On **myciel4**, SDGC achieves $\chi = 5$, matching DSATUR and improving upon greedy coloring by one color. The low edge density of this graph allows the BFS separation metric to locate same-colored vertices across meaningful hop distances, providing discriminative signal that guides color reuse toward structurally distant regions. The result demonstrates that SDGC can attain the chromatic number on sparse structured instances without enumerating the full color space.

On **DSJC125.1**, SDGC uses 18 colors, which is one above DSATUR (17) and three fewer than greedy (21). The separation and regularization components together reduce unnecessary color introductions compared to the greedy baseline, while the residual gap with respect to DSATUR reflects the advantage of saturation-degree ordering on irregular graphs: DSATUR's dynamic vertex selection consistently prioritizes the most constrained vertex at each step, a property that fixed degree-based ordering does not replicate.

On **queen15_15**, SDGC uses 28 colors, compared to 26 for greedy and 24 for DSATUR. This result reflects a known limitation of separation-driven reasoning under high density: when the average vertex degree is large, most candidate colors carry significant conflict penalties, and the hop distance to the nearest same-colored vertex is consistently small. In this regime, the conflict penalty $\beta \cdot \text{conf}(v, c)$ dominates the scoring function, the separation component provides limited discriminative value, and the new-color penalty $\gamma \cdot \text{new}(c)$ is insufficient to prevent new color introductions when existing colors are broadly conflicting. The result is a higher total color count than saturation-based methods, which remain effective in dense instances through their adaptive vertex ordering.

The expanded benchmark suite further clarifies the structural behavior of SDGC across multiple graph regimes. On sparse structured graphs such as anna and huck, SDGC consistently matches or approaches DSATUR while outperforming the standard greedy baseline. On medium-density random graphs including DSJC125.1 and DSJC250.1, the separation-aware scoring objective continues to reduce unnecessary color introduction relative to greedy coloring, although DSATUR retains a modest advantage due to its adaptive saturation-based ordering. The dense benchmark DSJC250.5 exhibits behavior similar to queen15_15, where conflict pressure dominates the scoring objective and the separation metric loses significant discriminative capability. In this regime, SDGC introduces more colors than DSATUR and, in some cases, more than greedy coloring. These results reinforce the conclusion that the framework is most effective on sparse and moderately dense instances where global color dispersion remains informative.

VI. REPRODUCIBILITY

All experiments were implemented in Python using the NetworkX library for graph construction and traversal, and NumPy for numerical operations. The DIMACS benchmark graphs were loaded from their standard binary format without modification. Vertex ordering in both SDGC and the greedy baseline is determined by decreasing degree, computed deterministically from the graph structure. DSATUR uses the standard saturation-degree ordering with tie-breaking by degree. All three algorithms are deterministic given fixed input; no random seeds are required. Additional DIMACS benchmark instances including anna, david, huck, DSJC250.1, and DSJC250.5 were evaluated using the same fixed parameter configuration. The complete source code, benchmark loading utilities, and scoring implementation are publicly available at [1]. Fixed parameter values used in all experiments are: $\alpha = 3$, $\beta = 10$, $\gamma = 8$, $k = 3$, repair iterations $= 3$.

VII. DISCUSSION

A. Effect of the Separation Metric

The BFS-based separation metric distinguishes SDGC from classical heuristics by introducing a form of global awareness into the assignment decision. Rather than relying solely on the immediate neighborhood, SDGC considers the proximity of same-colored vertices up to k hops away. This provides a richer signal for color selection in sparse and moderately dense graphs, where same-colored vertices are distributed across meaningful graph distances.

As edge density increases, however, the separation signal weakens. In dense graphs, the average shortest path between any two vertices is short, and same-colored vertices are likely to be reachable within the BFS depth bound regardless of the coloring strategy. The metric therefore loses discriminative power, and the conflict penalty increasingly governs the assignment outcome.

B. Role of Color Regularization

The new-color penalty $\gamma \cdot \text{new}(c)$ provides a structural bias toward color reuse that operates independently of local conflict information. Its primary effect is to reduce the frequency with

which the algorithm introduces a new color solely because the conflict landscape for existing colors is temporarily unfavorable. On medium-density instances such as DSJC125.1, this regularization contributes to a measurable reduction in total color count relative to the greedy baseline. On dense instances, the regularization effect is partially offset by the dominance of the conflict penalty, which limits the algorithm's ability to reuse existing colors without incurring violations.

C. Behavior on Dense Graphs

In high-density graphs, the conflict penalty $\beta \cdot \text{conf}(v, c)$ dominates the scoring function for the majority of vertices. With many neighbors already colored, most existing candidate colors carry non-trivial conflict penalties, and the new-color option—which carries zero conflict and maximum separation—tends to accumulate a higher score despite the regularization penalty. This structural imbalance leads to more frequent color introductions than saturation-based methods, which circumvent the problem through adaptive vertex ordering rather than a scored objective. As a result, SDGC's color count exceeds that of DSATUR on queen15_15, and the gap is attributable to the ordering strategy rather than the scoring function per se.

D. Unified Scoring and Algorithmic Consistency

A design property of SDGC is the application of a single scoring rule across all assignment stages. Hybrid methods that use different heuristics for the initial assignment and the repair phase risk introducing inconsistencies when the two stages optimize conflicting objectives. By using the same rule throughout, SDGC ensures that repair decisions are made on the same basis as initial ones, and that any modification to the scoring function—such as a change to the conflict penalty—propagates uniformly to all stages without requiring separate adjustment.

E. Trade-off Between Color Economy and Feasibility

The parameter γ governs the trade-off between color economy and constraint feasibility. A value that is too large suppresses color introduction to the point where the algorithm may be forced to assign infeasible colorings in highly constrained regions. A value that is too small fails to prevent unnecessary color proliferation. In the experiments reported here, $\gamma = 8$ provides a consistent balance across all three benchmarks, but this value may require adjustment for instances with substantially different density or constraint structure.

VIII. Limitations

SDGC has several limitations that constrain its applicability and should be considered when interpreting the experimental results.

The algorithm's performance is sensitive to the choice of α, β, and γ. While the parameter values used in this study ($\alpha = 3$, $\beta = 10$, $\gamma = 8$) produce consistent results across the evaluated benchmarks, there is no guarantee that these values are optimal for problem instances with different density, constraint structure, or chromatic number. Automatic parameter selection, for example through cross-validation on a held-out set of instances, would strengthen the practical applicability of the framework.

SDGC produces more colors than DSATUR on the dense queen15_15 benchmark, and the performance gap is likely to persist or widen on other dense instances. This is a structural limitation of the separation-aware objective under high edge density: the separation metric loses discriminative power, and the conflict penalty dominates in a regime where DSATUR's saturation-degree ordering retains its advantage. SDGC is therefore most competitive on sparse and moderately dense graphs where global color distribution provides actionable signal.

The BFS-based separation computation introduces overhead relative to purely local methods. Although bounded to depth k, each BFS call traverses a subgraph that can be large in dense instances, contributing to the higher runtime observed on queen15_15. Caching separation values between consecutive assignment steps or reducing k adaptively with graph density are potential directions for runtime improvement.

Future work may also explore the integration of SDGC's scoring objective with adaptive vertex ordering strategies, which could recover some of the performance gap on dense instances observed in the current evaluation.

IX. Conclusion

We presented Separation-Driven Graph Coloring (SDGC), a heuristic framework that formulates graph coloring as a scoring maximization problem. The scoring function integrates three components—a BFS-based separation metric, a constraint conflict penalty, and a color regularization term—into a unified objective applied consistently across all assignment stages. This design provides a principled mechanism for balancing global color dispersion, constraint feasibility, and color economy without requiring problem-specific heuristic modifications.

Evaluation on standard DIMACS benchmarks shows that SDGC matches DSATUR on sparse structured instances (myciel4), approaches it on medium-density instances (DSJC125.1), and produces valid but higher-color-count solutions on dense instances (queen15_15). The performance pattern across density levels is structurally interpretable: separation provides actionable signal when same-colored vertices are distributed across meaningful graph distances, and its influence diminishes as density increases and conflict pressure dominates.

The primary technical contribution of this work is the formalization of color regularization within a separation-aware scoring framework and the demonstration that a single unified rule can govern all assignment decisions without sacrificing competitive performance on sparse and moderately dense instances. These properties suggest that separation-aware scoring with regularization is a technically sound direction for heuristic graph coloring, particularly in applications where

global color distribution and constraint satisfaction must be jointly managed.

REFERENCES

[1] M. Lohchab, "SDGC: Separation-Driven Graph Coloring – Source Code," GitHub, 2024. [Online]. Available: https://github.com/Mann-lohchab/SDGC

[2] D. Brélaz, "New methods to color the vertices of a graph," *Communications of the ACM*, vol. 22, no. 4, pp. 251–256, 1979.

[3] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, 1979.

[4] D. J. A. Welsh and M. B. Powell, "An upper bound for the chromatic number of a graph and its application to timetabling problems," *The Computer Journal*, 1967.

[5] D. W. Matula, G. Marble, and J. D. Isaacson, "Graph coloring algorithms," in *Graph Theory and Computing*, 1972.

[6] M. Kubale, *Graph Colorings*. American Mathematical Society, 2004.

[7] F. T. Leighton, "A graph coloring algorithm for large scheduling problems," *Journal of Research of the National Bureau of Standards*, 1979.

[8] T. R. Jensen and B. Toft, *Graph Coloring Problems*. Wiley, 1995.

[9] S. Irnich and D. Villeneuve, "The shortest-path problem with resource constraints," *European Journal of Operational Research*, 2006.

[10] F. Roberts, "T-colorings of graphs: recent results and open problems," *Discrete Mathematics*, 1991.

[11] K. I. Aardal et al., "Models and solution techniques for frequency assignment problems," *Annals of Operations Research*, 2007.

[12] M. M. Halldórsson, "Wireless scheduling with power control," *ACM Transactions on Algorithms*, 2012.

[13] P. Gupta and P. R. Kumar, "The capacity of wireless networks," *IEEE Transactions on Information Theory*, 2000.

[14] J. Culberson, "Graph coloring page," 1992.

[15] A. Hertz and D. de Werra, "Using tabu search techniques for graph coloring," *Computing*, 1987.